

# Deep Reinforcement Learning for Thai Stock Portfolio Allocation

## Comprehensive 100-Slide Project Summary

### Final Project Progress Report

BistKA HPC Project: `/lustrefs/project/25sfcs03/drl_thai_stock`

June 10, 2026

# How To Read This Deck

- This deck explains what has been built, tested, and learned.
- It uses simple language first, then adds technical details.
- The results are pilot results, not final market-performance claims.
- The deck is organized from motivation to data, model, results, and next steps.

# Project In One Sentence

- The project builds a reproducible research pipeline for testing reinforcement learning portfolio allocation on Thai stocks.
- It compares DRL agents with simple baselines.
- It adds multi-source features step by step.
- It runs and validates the work on BistKA HPC.

# Main Research Question

- Can a DRL portfolio agent improve long-term profit and risk control for Thai stocks?
- Can it beat simple baselines such as equal weight and buy-and-hold?
- Can extra information such as index, sector, macro, fundamentals, and sentiment help?
- Can the experiment be made reproducible enough to trust the result?

# Why This Project Matters

- Financial ML experiments can look good if data leakage or weak baselines are ignored.
- This project focuses on careful experimental wiring.
- It records data provenance and validation status.
- It shows what works now and what still needs better data.

# Final Status At A Glance

- Local repository scaffold is complete.
- HPC project directory and conda environment are complete.
- Latest full local validation passed with 53 tests on Python 3.11.
- External data handoff, intake, and validation gates are implemented.
- Latest documentation and validation artifacts are synced to BistKA.

# Important Caveat

- The current real-data experiments use a five-stock starter universe.
- These results validate the pipeline, not the whole Thai market.
- Historical sentiment and official macro data do not yet overlap the 2021-2024 model window.
- Stronger performance claims require broader and licensed data.

# What Has Been Built

- Data ingestion for synthetic and real OHLCV data.
- Feature engineering for technical and contextual features.
- A Gymnasium-compatible trading environment.
- DRL training, baselines, evaluation, plots, and result aggregation.
- HPC job scripts, external-data gates, and repeatable validation commands.

# Main Project Folder

- HPC working folder:
  - `/lustrefs/project/25sfcs03/drl_thai_stock`
- Local working folder:
  - `/Users/nathawat/Documents/final_project_plan`
- GitHub remote:
  - `github.com/pppppppppppp-North/drl`

# Current Repository Shape

- `config/`: experiment settings.
- `src/`: ingestion, features, environment, training, experiments.
- `tests/`: unit and pipeline tests.
- `docs/`: methodology, data sources, and artifact notes.
- `slurm/`: HPC job templates.

# HPC Setup Completed

- BistKA project directory was created.
- Python environment was created at `/lustrefs/project/25sfcs03/envs/dr1_env`.
- Code, configs, docs, tests, and selected outputs were synced to HPC.
- File-separation snapshots track uploaded files versus generated files.

# Latest HPC Validation

- Latest full HPC test suite: 53 tests passed.
- Ticker-universe focused tests: 4 tests passed.
- Historical sentiment importer tests: 3 tests passed.
- Official macro CSV importer tests: 3 tests passed.
- Source-readiness and intake-validation gates run on BistKA.

# Project Phases

- ① Build a reproducible scaffold.
- ② Add synthetic pilot data.
- ③ Add real OHLCV starter data.
- ④ Build trading environment and baselines.
- ⑤ Train and compare DRL agents.
- ⑥ Add multi-source features and validation studies.

# Synthetic Pilot Purpose

- Synthetic data was used first to test the code safely.
- It avoids waiting for real data before checking the pipeline.
- It verifies that training, evaluation, and plots can run end to end.
- It is not used for final market conclusions.

# Synthetic Data Generator

- Implemented deterministic synthetic price generation.
- Generates daily stock-like OHLCV rows.
- Includes placeholder market return, macro change, and sentiment score.
- Useful for tests and smoke runs.

- The real OHLCV starter universe has five Thai tickers.
- ADVANC.BK, AOT.BK, CPALL.BK, KBANK.BK, PTT.BK.
- Data source is Yahoo Finance through `yfinance`.
- Licensed SET or SETSMART data is still preferred for final publication.

# Real OHLCV Feature Table

- Feature file: `data/processed/features_real_ohlcw.parquet`.
- Rows: 4,840.
- Tickers: 5.
- Columns: 21.
- Date range: 2021-01-04 to 2024-12-30.

# Train Validation Test Split

| Split      | Date range               | Rows | Dates |
|------------|--------------------------|------|-------|
| Train      | 2021-01-04 to 2023-05-29 | 2900 | 580   |
| Validation | 2023-05-30 to 2024-03-12 | 970  | 194   |
| Test       | 2024-03-13 to 2024-12-30 | 970  | 194   |

- The split is chronological.
- Future data is not mixed into earlier training periods.

# Data Quality Checks

- Data-quality reports were generated for the real feature tables.
- Ticker coverage and missingness are tracked.
- The real OHLCV starter table has complete five-ticker panel coverage.
- Leakage checks verify duplicate rows and split boundaries.

- `data/data_manifest.csv` records source provenance.
- It records access method, frequency, date range, symbols, columns, and missing-value rate.
- It separates synthetic, Yahoo, BOT, and user-provided import paths.
- A provider manifest template now supports future licensed-data intake.

# Ticker Universe CSV Support

- Added `src/data/ticker_universe.py`.
- Real ingesters can now read tickers from a CSV or text file.
- This avoids hard-coding all tickers in YAML.
- It prepares the project for a broader SET50 or licensed universe file.

# Starter Universe CSV

- Added `data/reference/thai_starter_universe.csv`.
- It contains the current five starter tickers.
- Added example config `config/real_ohlcv_universe_csv.yaml`.
- The same mechanism can load a larger curated ticker list later.

# Technical Features

- 1-day, 5-day, and 20-day returns.
- 20-day rolling volatility.
- 10-day and 30-day moving-average ratios.
- RSI, MACD, Bollinger z-score, ATR, and volume ratio.

# Why Technical Features Are Used

- Raw prices alone are hard for a model to interpret.
- Returns describe recent price movement.
- Volatility describes risk.
- Momentum indicators describe trend and overbought or oversold behavior.

# SET Market Context

- Added SET index ingestion using `^SET.BK`.
- Raw file: `data/raw/prices_market_indices.csv`.
- Merged file: `features_real_ohlcv_market.parquet`.
- This gives each stock a market-wide context.

# SET Index Features

- `set_return_1d.`
- `set_return_5d.`
- `set_return_20d.`
- `set_volatility_20d.`
- `set_ma_ratio_20.`

- Added pilot sector mapping for the five starter tickers.
- Mapping file: `data/reference/sector_mapping_thai_pilot.csv`.
- Features include peer return, relative return, peer count, and sector flags.
- This is a pilot fallback until better sector-index data is available.

| Ticker    | Sector         |
|-----------|----------------|
| ADVANC.BK | ICT            |
| AOT.BK    | Transportation |
| CPALL.BK  | Commerce       |
| KBANK.BK  | Banking        |
| PTT.BK    | Energy         |

- Added Yahoo sector-index candidate probe.
- Tested 139 candidate symbols on HPC.
- Only banking symbol ^BANK was usable.
- This proved that a complete public Yahoo sector-index source was not available.

# Macro Proxy Features

- Added Yahoo Finance macro proxy ingestion.
- Series: USDTHB, Brent, WTI, gold, and US 10-year yield.
- Raw file: `data/raw/prices_macro_yahoo.csv`.
- These are proxies, not official Thai macro releases.

# Macro Proxy Merge

- Macro proxy values are merged by date.
- Merge uses forward fill from the latest known observation.
- This avoids using future observations.
- Merged feature table has 4,840 rows and 68 columns.

# Fundamental Features

- Added Yahoo annual and quarterly financial statement ingestion.
- Raw file has 8,701 statement rows.
- Coverage spans five tickers and 13 period ends.
- Features are merged only after a 60-day reporting lag.

# Why A Reporting Lag Matters

- A financial statement is not known on the exact period-end date.
- Using it too early would create lookahead bias.
- The project delays fundamentals by 60 days.
- This is a conservative rule for pilot experiments.

# Official Macro Source Probe

- Added Bank of Thailand source probe.
- Targeted BOTWEBSTAT Leading Economic Indicator table.
- Raw file: `data/raw/bot_official_macro.csv`.
- Release-date-aware feature merge is implemented.

# Official Macro Limitation

- BOT web table probe returned rows dated 2025-11-01 to 2026-04-01.
- The modeling window is 2021-2024.
- Therefore the probe has no historical overlap.
- No official macro performance claim is made yet.

# Official Macro CSV Importer

- Added `src/data/ingest_official_macro_csv.py`.
- Supports long and wide CSV formats.
- Can assign release dates when a source file lacks them.
- This lets a downloaded historical BOT export enter the pipeline later.

- Added Yahoo latest-news probe.
- Raw file: `data/raw/news_yahoo_latest.csv`.
- It returned only 11 rows.
- Dates were 2025-06-24 to 2026-03-31, outside the model window.

# Sentiment Merge Scaffold

- Daily sentiment extraction is implemented.
- Sentiment is merged backward-only by ticker and date.
- Maximum news age is seven days.
- This prevents future news from leaking into earlier stock rows.

# Historical Sentiment CSV Importer

- Added `src/data/ingest_sentiment_csv.py`.
- It normalizes user-provided historical news or sentiment CSV files.
- It can produce event-level news and daily ticker sentiment.
- It waits for a valid historical source instead of fabricating data.

# No-Lookahead Principle

- Features must only use information known at or before the trading date.
- Market and macro proxies are backward or forward-filled from known history.
- Fundamentals use a reporting lag.
- News and official macro data use release dates.

# Trading Environment

- Implemented `ThaiStockTradingEnv`.
- It follows the Gymnasium API.
- The agent chooses long-only portfolio weights.
- The environment tracks portfolio value through time.

- The action is a vector of weights, one per stock.
- Weights are normalized into a long-only budget.
- Short selling is not used in this pilot.
- Cash weight is tracked separately.

# Observation Space

- The environment gives the model a lookback window.
- The window contains the selected feature columns.
- This creates a stable tensor shape for DRL algorithms.
- Tests verify reset and step observation shapes.

- The environment applies transaction costs.
- It tracks net return, turnover, and drawdown.
- It records per-ticker action weights.
- These values are saved for evaluation and plots.

# Reward Function

- Reward combines return, turnover penalty, and drawdown penalty.
- Current real-data settings:
- `return_weight = 1.0`
- `turnover_penalty = 0.05`
- `drawdown_penalty = 0.1`

# Why Penalize Turnover

- High turnover means frequent portfolio changes.
- Frequent trading causes transaction costs.
- A turnover penalty discourages noisy trading.
- It makes the reward more realistic.

# Why Penalize Drawdown

- Drawdown measures loss from a previous peak.
- Investors care about large losses, not only final return.
- A drawdown penalty encourages risk control.
- This is important for long-term portfolio optimization.

# Implemented DRL Algorithms

- PPO is the main algorithm.
- A2C is implemented and compared.
- Support paths exist for DDPG, TD3, and SAC.
- PPO and A2C were the main validated comparison models.

# Why PPO

- PPO is widely used for continuous-control RL problems.
- It is more stable than many older policy-gradient methods.
- It works well with vectorized environments.
- It is a reasonable first DRL baseline for portfolio allocation.

# Baselines Implemented

- Equal weight.
- Random allocation.
- Momentum.
- Buy-and-hold per ticker.
- Moving-average crossover.
- Mean-variance.

# Why Baselines Matter

- A DRL model is only useful if it beats simple alternatives.
- Buy-and-hold is a strong and honest benchmark.
- Equal weight is simple, transparent, and hard to overfit.
- Baselines keep the research conclusion grounded.

# Training Outputs

- Saved model files.
- Equity curves.
- Metric CSV files.
- Action-weight logs.
- TensorBoard logs and plot-ready outputs.

# Evaluation Metrics

- Cumulative return.
- Annualized return.
- Sharpe ratio.
- Maximum drawdown.
- Final portfolio value.
- Turnover and active trading steps.

# Plotting Utility

- Implemented result plotting utility.
- It generates equity-curve plots.
- It generates drawdown plots.
- It generates turnover and action-weight plots.
- These are useful for report-ready figures.

# Synthetic Comparison Result

| Policy            | Cum. return | Sharpe |
|-------------------|-------------|--------|
| buy_hold_KBANK.BK | 0.580       | 3.219  |
| ma_crossover      | 0.258       | 1.881  |
| equal_weight      | 0.153       | 1.398  |
| a2c               | 0.167       | 1.373  |
| ppo               | -0.013      | -0.108 |

# Synthetic Result Interpretation

- Synthetic data confirmed the whole pipeline runs.
- Some simple baselines beat PPO in the synthetic comparison.
- This is acceptable because synthetic data is only a test bed.
- The important result is that training, evaluation, and aggregation worked.

# Real OHLCV Main Comparison

| Policy             | Cum. return | Sharpe | Max DD |
|--------------------|-------------|--------|--------|
| buy_hold_PTT.BK    | 0.044       | 0.405  | -0.103 |
| buy_hold_ADVANC.BK | 0.010       | 0.097  | -0.099 |
| equal_weight       | -0.031      | -0.417 | -0.103 |
| a2c                | -0.038      | -0.434 | -0.120 |
| ppo                | -0.036      | -0.440 | -0.066 |

- Best validation policy was buy-and-hold PTT.
- PPO and A2C did not beat the strongest simple baseline.
- PPO had smaller max drawdown than A2C and equal weight.
- This is a useful and honest pilot outcome.

# Invalidated Run

- Run `real_ohlcv_compare_2854` was invalidated.
- Reason: an old training entrypoint regenerated synthetic data into real paths.
- Corrected valid run is `real_ohlcv_compare_2855`.
- This was recorded clearly in the checklist and manifest.

# Hyperparameter Search

- Added Optuna-style trial runner and SLURM array workflow.
- Search space includes learning rate, gamma, GAE lambda, entropy coefficient, rollout length, clip range, and batch size.
- Fixed search run: `optuna_search_fixed_2863`.
- Earlier repeated-parameter search was invalidated.

## Best Short Optuna Trial

| Trial | Algo | Sharpe | Cum. return | Max DD |
|-------|------|--------|-------------|--------|
| 4     | PPO  | 0.280  | 0.020       | -0.069 |
| 0     | PPO  | -0.306 | -0.003      | -0.009 |
| 1     | A2C  | -0.520 | -0.043      | -0.127 |
| 2     | PPO  | -1.080 | -0.089      | -0.115 |
| 3     | A2C  | -1.708 | -0.003      | -0.004 |

# Longer Tuned PPO Run

- Best short PPO trial was promoted into a longer config.
- Longer run used 20,000 timesteps.
- Result: cumulative return -0.030 and Sharpe -0.403.
- The tuned parameters were not robust enough yet.

# Walk-Forward Validation

- Added rolling-window split generation.
- Starter settings: 504 training dates, 126 validation dates, 126 test dates.
- Step size is 126 dates.
- First pilot ran two short PPO windows on HPC.

# Walk-Forward Pilot Results

| Policy             | Mean cum. return | Mean Sharpe |
|--------------------|------------------|-------------|
| buy_hold_ADVANC.BK | 0.084            | 1.468       |
| buy_hold_KBANK.BK  | 0.064            | 1.157       |
| buy_hold_PTT.BK    | 0.041            | 0.672       |
| equal_weight       | 0.021            | 0.633       |
| ppo                | 0.004            | 0.058       |

# Walk-Forward Interpretation

- Walk-forward machinery is verified.
- The first PPO pilot produced small positive average test return.
- Simple buy-and-hold policies still performed better.
- More windows and richer data are needed for stronger conclusions.

# Ablation Study Purpose

- Ablations test which feature groups help.
- They prevent guessing about feature value.
- The pilot compared returns-only, technical, and technical-plus-context configs.
- It used short 1,000-timestep PPO jobs.

# PPO Ablation Summary

| Feature group       | Cum. return | Sharpe |
|---------------------|-------------|--------|
| Returns only        | -0.010      | -0.453 |
| Technical + context | -0.056      | -0.824 |
| Technical only      | -0.079      | -1.611 |

# Ablation Interpretation

- Ablation machinery is verified.
- Short PPO ablations did not beat buy-and-hold PTT.
- Returns-only PPO was least negative among the three PPO ablations.
- Stronger claims need broader data and longer validation.

# Regime And Stress Testing

- Added regime/stress-test runner.
- It evaluates baselines plus saved PPO/A2C models.
- It uses named market slices and high-volatility windows.
- Output includes slices, metrics, equity curves, and summary report.

- Acute 2020 COVID crash cannot be evaluated.
- Reason: current real OHLCV starts on 2021-01-04.
- The available COVID-related slice is COVID recovery in 2021.
- This limitation is explicitly recorded.

## Best Policy By Regime

| Regime              | Best policy     | Sharpe |
|---------------------|-----------------|--------|
| COVID recovery 2021 | ADVANC buy-hold | 2.376  |
| High volatility 1   | ADVANC buy-hold | -0.735 |
| High volatility 2   | A2C             | 7.221  |
| Inflation hike 2022 | AOT buy-hold    | 1.367  |
| Validation period   | PTT buy-hold    | 0.405  |

# Market-Context Pilot

- First short SET market-context validation run completed.
- Output directory: `real_ohlcv_market_pilot_20260531`.
- It verifies SET-index feature ingestion and merge.
- It still does not solve sector-relative data.

- First short sector-context validation run completed.
- Output directory: `real_ohlcv_sector_pilot_20260531`.
- PPO improved over the SET-market-context PPO pilot.
- It still did not beat the PTT buy-and-hold baseline.

- First short macro-proxy validation run completed.
- Output directory: `real_ohlcv_macro_pilot_20260531`.
- Macro-proxy PPO improved over sector-context PPO.
- It beat equal weight but not the strongest buy-and-hold baselines.

- First short fundamentals validation run completed.
- Output directory: `real_ohlc_fundamentals_pilot_20260531`.
- PPO had lower drawdown than equal weight.
- But it still did not beat PTT or ADVANC buy-and-hold.

# Sentiment Scaffold Status

- News probe, daily scoring, and merge paths are implemented.
- Historical sentiment CSV importer is implemented and tested.
- Current Yahoo latest news has no 2021-2024 overlap.
- A valid historical news source is still needed.

# Official Macro Scaffold Status

- BOT source probe and release-date merge are implemented.
- Historical official macro CSV importer is implemented and tested.
- Current BOT web rows have no 2021-2024 overlap.
- A historical official macro export is still needed.

# Testing Strategy

- Unit tests cover environment behavior and feature logic.
- Ingestion tests cover source normalization.
- Experiment tests cover aggregation and config logic.
- HPC full-suite validation is used because local Python lacks some dependencies.

# Latest Test Progress

| Milestone                         | Tests passed |
|-----------------------------------|--------------|
| Sector-index probe                | 43           |
| Official macro CSV importer       | 46           |
| Historical sentiment CSV importer | 49           |
| Ticker-universe CSV support       | 53           |

# File Separation On HPC

- User asked to separate uploaded files from files already on HPC.
- Added `scripts/file_separation_snapshot.py`.
- Latest snapshot: `_file_separation/20260531_231548`.
- This helps distinguish code uploads from generated outputs.

- Latest archive:
- `final_archive/20260531_final_project_pilot_with_ticker_universe_csv.tar.gz`.
- It contains source, configs, tests, docs, selected data, reports, and selected result outputs.
- It intentionally excludes large training caches and TensorBoard event files.

- Project was pushed to GitHub.
- Recent commits include full external config, provider request, intake checklist, and intake validator.
- Latest pushed commit before this slide refresh: 5eaab95.
- Remote branch: `main`.

# Main Engineering Lesson

- The pipeline matters as much as the model.
- Without clean data splits, leakage checks, and baselines, DRL results are easy to overstate.
- This project now has a reliable experimental skeleton.
- The next bottleneck is data coverage, not only model code.

# Main Modeling Lesson

- PPO and A2C did not beat the strongest simple baselines in the starter experiment.
- This does not mean DRL is useless.
- It means the current data and feature set are not enough to justify a strong DRL claim.
- The result is still useful because it sets a realistic benchmark.

# What Worked Well

- End-to-end training and evaluation works.
- Baselines are implemented and comparable.
- Multi-source feature pipelines are modular.
- HPC testing and archiving workflow is repeatable.
- Limitations are documented instead of hidden.

# What Is Still Missing

- Broader curated or licensed stock universe.
- Complete sector-index or constituent source.
- Historical official macro source with release dates.
- Historical Thai news or sentiment source.
- Licensed fundamentals with stronger coverage.
- These are now explicit provider-file blockers, not unknown code tasks.

# Why Results Are Called Pilot Results

- Only five real tickers are active.
- Some feature groups are scaffolds because historical source data is missing.
- Short 1,000-timestep pilots verify wiring, not final performance.
- The project is ready for stronger experiments once data improves.

# Data Sources Completed

- Synthetic pilot prices.
- Yahoo real OHLCV starter data.
- Yahoo SET index data.
- Yahoo macro proxy data.
- Yahoo statement fundamentals.
- Yahoo latest-news probe.
- BOT official macro probe.
- External data templates for the five missing provider files.

# Import Paths Completed

- OHLCV ingestion.
- Market index ingestion.
- Macro proxy ingestion.
- Fundamentals ingestion.
- Latest-news probe.
- Historical sentiment CSV import.
- Historical official macro CSV import.
- Ticker-universe CSV import.
- External intake validation for provider-file packages.

# Experiment Paths Completed

- Single training run.
- Algorithm comparison.
- Baseline evaluation.
- Hyperparameter search.
- Walk-forward pilot.
- Ablation pilot.
- Regime and stress tests.

# Report Artifacts Completed

- Final methodology/results markdown report.
- Final methodology/results LaTeX report.
- Comprehensive introduction/literature-review PDF.
- External data handoff, provider request, and intake checklist.
- Data source documentation.
- Final artifact manifest.
- Intake validation CSV and Markdown reports.

# How To Explain The Result Simply

- The system works.
- The model is not yet better than simple buy-and-hold.
- The experiment is honest about this.
- The next improvement should focus on better and broader data.

## Recommended Next Step 1

- Replace the five-ticker starter universe with a broader curated list.
- Save it at `data/reference/set50_or_set100_universe.csv`.
- Run source-readiness and intake-validation gates.
- Only then rebuild OHLCV and feature tables.

## Recommended Next Step 2

- Replace pilot sector mapping with a complete sector or constituent source.
- Then rebuild sector-relative features.
- This will make sector-relative signals more meaningful.
- It will also support broader universe experiments.

## Recommended Next Step 3

- Obtain historical official macro data with release dates.
- Use the official macro CSV importer.
- Rebuild official macro feature tables.
- Only then run official macro ablations.

## Recommended Next Step 4

- Obtain historical Thai news, disclosures, or sentiment scores.
- Use the historical sentiment CSV importer.
- Build daily ticker-level sentiment.
- Re-run sentiment feature merge and ablations.

## Recommended Next Step 5

- Follow `docs/full_external_rebuild.md`.
- Produce CSV and Markdown intake-validation reports.
- Run broader ablations, walk-forward validation, and regime tests.
- Report average, per-window stability, and failure cases.

# Final Takeaway

- The project now has a working, tested, reproducible pilot pipeline.
- It includes data ingestion, features, DRL training, baselines, and evaluation.
- Current DRL results do not beat the strongest simple baseline.
- This is a strong foundation for the next data-driven experiment.

# What To Say In A Presentation

- "We built the full experimental system first."
- "We tested it on synthetic data and real Thai starter data."
- "We compared DRL with simple but strong baselines."
- "The honest result is that better data is needed before claiming DRL superiority."

- Thank you.
- Full project code, configs, tests, reports, and archive paths are documented.
- Latest verified full test count: 53 passed.
- Current blocker: five external provider files.
- Current status: reproducible pilot plus complete data handoff workflow.